Documentazione Tecnica del Progetto regular_clone

Allineamento codice-runtime e metodologia produttiva

Team Progetto

23 Marzo 2026

Indice

1 Document Control **4**
1.1 Revision History . . . 4
1.2 Scopo del documento . . . 5
1.3 Perimetro e policy . . . 5

2 Executive Technical Overview **6**
2.1 Obiettivo del sistema . . . 6
2.2 Modalita operative . . . 6
2.3 Come funziona in modo semplice (per investitori) . . . 6
2.4 KPI ufficiali . . . 7
2.5 Allineamento metodologico di alto livello . . . 7

3 Assunzioni Sperimentali (Baseline Pre-Calibrazione) **8**
3.1 Policy assunzioni . . . 8
3.2 Tabella master assunzioni indica/sativa . . . 8
3.3 Nota di lettura . . . 9

4 Architettura Software **10**
4.1 Layer applicativi . . . 10
4.2 Data flow end-to-end . . . 10
4.3 Mappa zone capannone vs dominio software . . . 11

5 Parametrizzazione Fenologica e Spazio Decisionale (Dettaglio Operativo) **12**
5.1 Fasi fenologiche . . . 12
5.2 Vincoli varietali e produttivi runtime . . . 12
5.3 Vincoli hard pre-ciclo (seme -> talea) . . . 12
5.4 Spazio parametrico ottimizzato (literature baseline) . . . 13
5.5 Valori sperimentali assunti: rimando . . . 13
5.6 Override genetici/famiglia/cultivar . . . 13
5.7 Evidenze bibliografiche in registry . . . 14

6 Formulazione Tecnico-Matematica del Modello **15**
6.1 Variabili derivate . . . 15
6.2 Dinamica giornaliera . . . 15
6.3 Funzioni di risposta . . . 15
6.4 Pesi fenologici e gain . . . 15
6.5 Penalty e rischio fitosanitario . . . 16

7 Controller Adattivo Closed Loop **17**
7.1 Target VPD fase-specifici . . . 17
7.2 Regole di controllo implementate . . . 17
7.3 Interazione con quality-gate e fallback . . . 17

8 Digital Twin: Dettagli Implementativi **18**
8.1 Ciclo di simulazione . . . 18
8.2 Robustezza numerica e guardrail . . . 18
8.3 Parametri di calibrazione twin . . . 18
8.4 Output strutturato . . . 19

9 Constrained Bayesian Optimization **20**
9.1 Obiettivi e vincoli . . . 20
9.2 Violation function . . . 20
9.3 Modello probabilistico multi-GP . . . 20
9.4 Acquisition e fattibilita . . . 20
9.5 Strategia restart e robustezza . . . 21

10 Contratto Dati Realtime e Integrazione I/O **22**
10.1 Campi obbligatori e opzionali payload sensore . . . 22
10.2 Sorgenti e sink supportati . . . 22
10.3 Schema output `control_tick` . . . 23

11 Runbook Operativo **24**
11.1 Setup ambiente . . . 24
11.2 Calibrazione twin . . . 24
11.3 Ottimizzazione offline . . . 24
11.4 Realtime . . . 24
11.5 Analisi economica pre-go-live . . . 25
11.6 Baseline startup low-cost ufficiale . . . 25
11.7 Artefatti runtime . . . 26

12 Validazione e Test **27**
12.1 Strategia test . . . 27
12.2 Suite test corrente . . . 27
12.3 Criteri minimi di accettazione . . . 28

13 Sicurezza Operativa **29**
13.1 Guardrail minimi . . . 29
13.2 Fault handling e policy fallback . . . 29
13.3 Alerting rolling-window . . . 29

14 Traceability e Governance **31**
14.1 Matrice codice -> sezioni . . . 31
14.2 Run identity e firme . . . 31
14.3 Persistenza auditabile . . . 31

15 Allineamento Metodologico al Business Plan **33**
15.1 Trasposizione metodologia produttiva . . . 33
15.2 Scostamenti espliciti BP vs codice . . . 33
15.3 Gap metodologici dichiarati . . . 34
15.4 Implicazioni ingegneristiche degli scostamenti . . . 34

16 Limiti Noti e Roadmap **35**
16.1 Limiti attuali . . . 35
16.2 Roadmap tecnica suggerita . . . 35

17 Riferimenti Tecnici **36**
17.1 Fonti runtime e interne 36
17.2 Letteratura agronomica in registry 36
17.3 Riferimenti metodologici software/matematici 36

A Esempio payload sensore **37**

B Esempio output control_tick **38**

C Requisiti software **39**

Capitolo 1

Document Control

1.1 Revision History

Versione	Data	Autore	Descrizione
1.9	2026-03-23	Team progetto	Introduzione baseline startup low-cost ufficiale con selezione min-CAPEX vincolata (`startup_low_capex + grid_only_retrofit + core_efficiency_extended`), quality floor nativo `62` su modalita resa, telemetria energetica opzionale `power_*` nei control tick e aggiornamento runbook/documentazione business.
1.8	2026-03-23	Team progetto	Aggiornata semantica resa annua da vincolo `<=` a target esatto `=` con pianificazione area attiva necessaria; aggiornati optimizer/realtime/planner/economics, test e memo business.
1.7	2026-03-23	Team progetto	Ottimizzazione flusso editoriale: nuovo capitolo dedicato alle assunzioni sperimentali con tabella master (`hard runtime/calibrabile`), KPI economici conservativi su resa pianificata e inserimento memo business investitore in `docs/business`.
1.6	2026-03-23	Team progetto	Inserita sezione esplicita sui valori sperimentali assunti pre-calibrazione (fenologia, coefficienti genetici/famiglia, bounds calibrazione twin) e riorganizzazione path repository (`docs/*`, `economics/*`).
1.5	2026-03-23	Team progetto	Hardening pre-calibrazione: template dataset self-contained e family-coherent, rimozione variabili durata non piu ottimizzabili (`veg_days/flower_*_days`), KPI economici estesi (ricavi, EBITDA, ROI, break-even) e script operativo `scripts/run_economic_analysis.py`.

1.4	2026-03-22	Team progetto	Enforcement hard del cap energia in realtime anche con `-profile-json` (`controller_literature_realtime.py` e `core/realtime_io.py`); aggiornamento sezione test/coerenza.
1.3	2026-03-22	Team progetto	Introduzione vincolo hard `yield_cap_annual_kg` in optimizer/realtime, planner default a 80 kg/anno, integrazione modulo economico locale `economics/cea_economic_analysis.py`.
1.2	2026-03-22	Team progetto	Supporto nativo realtime per `max_yield_energy`/`max_quality_energy` e sezione esplicativa ad alto livello per stakeholder non tecnici.
1.1	2026-03-22	Team progetto	Riorganizzazione capitolo fenologico: parametri `10-14 gg germinazione` e `27-35 gg seed->talea` dichiarati hard-coded con evidenza implementativa.
1.0	2026-03-22	Team progetto	Prima emissione tecnica dedicata al runtime `regular_clone`, con policy codice dominante e mappa scostamenti dal business plan.

1.2 Scopo del documento

Questo documento definisce l'architettura tecnica, la formulazione tecnico-matematica e le procedure operative del progetto `regular_clone`.
La fonte implementativa primaria e il codice del repository corrente (`regular_clone/*`).
La fonte metodologica di riferimento e il documento interno *BUISNESS PLAN (1)*.

1.3 Perimetro e policy

- **Policy di allineamento**: codice dominante.
- **Uso del business plan**: riferimento metodologico per ciclo da talea, schema impiantistico DWC/NFT, gestione soluzione nutritiva e zoning operativo.
- **Scostamenti**: riportati esplicitamente nel Capitolo 14.

Capitolo 2

Executive Technical Overview

2.1 Obiettivo del sistema

Il sistema implementa due anelli integrati:

1. **Anello strategico offline**: ottimizzazione constrained Bayesian (`optimizer_literature_best.py`) su digital twin.
2. **Anello operativo realtime**: controllo adattivo closed-loop (`controller_literature_realtime.py`) con quality gate, fallback di sicurezza e supervisione MPC opzionale.

2.2 Modalita operative

Modalita supportate dall'optimizer e dal controller realtime:

- `max_yield`
- `max_quality`
- `max_yield_energy`
- `max_quality_energy`

Le varianti `*_energy` introducono un vincolo hard su energia per ciclo (`energy_cap_kwh_m2`, default 700.0, configurabile da CLI).
Tutte le modalita introducono inoltre un vincolo hard di resa annua farm-level (`yield_target_annual_kg`, alias CLI compatibile `yield_cap_annual_kg`, default 80.0).
Per le modalita orientate alla resa (`max_yield`, `max_yield_energy`) e attivo un guardrail di qualita minimo (`quality_floor`, default 62), verificato sia in ottimizzazione sia in realtime anche in caso di profilo esterno caricato con `-profile-json`.
Dato il profilo ottimizzato, il runtime calcola l'area attiva pianificata necessaria per rispettare il target esatto:

$$\texttt{planned_active_area_m2_for_target} = \frac{\texttt{yield_target_annual_kg}}{\texttt{dry_yield_kg_m2_year}}$$

e valida $\texttt{planned_active_area_m2_for_target} \leq \texttt{farm_active_area_m2}$.
Nel loop di controllo, le modalita `*_energy` usano la stessa logica closed-loop della modalita base corrispondente (`max_yield` oppure `max_quality`), ma operano su un profilo ottimizzato con vincolo energetico. Se viene passato un profilo esterno (`-profile-json`), il runtime valida comunque in avvio che `energy_kwh_m2_cycle <= energy_cap_kwh_m2` per le modalita `*_energy`.

2.3 Come funziona in modo semplice (per investitori)

Il sistema puo essere letto come una piattaforma di gestione rischio-rendimento per una farm idroponica:

1. si calibra il modello su una postazione pilota, per allineare simulazione e comportamento reale;
2. si genera una ricetta ottimale di ciclo (resa o qualita), con eventuale tetto energetico compatibile col budget;
3. si applica un target hard di produzione annua in uguaglianza per mantenere la farm entro il budget/obiettivo commerciale;
4. in produzione i sensori misurano ogni giorno (o tick) lo stato reale della coltura e dell'ambiente;
5. il controller confronta reale e target, calcola correzioni operative e invia comandi agli attuatori;
6. ogni decisione e tracciata (DB + log), cosi da auditare performance, sicurezza e costi energetici.

In termini economici, il valore e trasformare un processo agricolo complesso in una procedura controllabile, misurabile e ripetibile, con compromesso esplicito tra produttivita e consumo energia. Per una lettura business non tecnica, fare riferimento al memo investitore `docs/business/BUSINESS_MEMO_REGULAR_CLONE_IT.pdf`.

2.4 KPI ufficiali

KPI	Campo runtime	Significato
Resa areale	`dry_yield_g_m2`	Produzione secca per m2 a fine ciclo.
Qualita sintetica	`quality_index`	Indice 0–100 derivato da segnali spettro/stress/salute.
Energia ciclo	`energy_kwh_m2`	Energia per m2 per ciclo (LED + HVAC proxy).
Efficienza energetica	`g_per_kwh`	Resa secca per kWh.
Rischio processo	`penalty`	Penalita da violazioni VPD, pH, EC, DO, RH.
Rischio fitosanitario	`disease_pressure`, `hlvd_pressure`	Proxy cumulativi sanitari.
Robustezza BO	`feasibility_violation`, `uncertainty`	Coerenza vincoli e dispersione multi-run.

2.5 Allineamento metodologico di alto livello

Elementi del business plan direttamente riflessi nel runtime:

- ciclo produttivo da talea;
- `propagation_system = DWC`;
- `production_system = NFT_recirculating`;
- fotoperiodicita controllata in fase di fioritura.

Capitolo 3

Assunzioni Sperimentali (Baseline Pre-Calibrazione)

3.1 Policy assunzioni

Prima della calibrazione su dati pilota, il runtime usa un set di assunzioni sperimentali codificate. Ogni assunzione e classificata come:

- **hard runtime**: vincolo non ottimizzabile nel loop giornaliero;
- **calibrabile**: parametro modificabile in fase di fit (`calibrate_twin.py`).

3.2 Tabella master assunzioni indica/sativa

Parametro	Indica	Sativa	Stato	Fonte implementativa
`clone_cycle_days_from_cutting`	105	120	hard runtime	`core/model.py`
`stage_days_from_cutting`	12/24/7/32/30	12/27/7/37/37	hard runtime	`core/model.py`
`flower_density_pl_m2`	4.0	2.0	hard runtime	`core/model.py`, `core/production_planner.py`
`seed_germination_days`	10–14	10–14	hard runtime	`SEED_GERMINATION_DAYS_RANGE`
`seed_to_cutting_days`	27–35	27–35	hard runtime	metadata profilo regular
`yield_potential_scale`	1.99 (fam.)	1.02 (fam.)	calibrabile (indiretto)	`configs/genetic_profiles.json`
`quality_potential_scale`	0.98 (fam.)	1.03 (fam.)	calibrabile (indiretto)	`configs/genetic_profiles.json`
`disease_pressure_scale`	0.97 (fam.)	1.03 (fam.)	calibrabile (indiretto)	`configs/genetic_profiles.json`
`hlvd_pressure_scale`	1.00 (fam.)	1.01 (fam.)	calibrabile (indiretto)	`configs/genetic_profiles.json`
`TwinCalibration.yield_gain_scale`	0.60–1.60	0.60–1.60	calibrabile (diretto)	`calibrate_twin.py`
`TwinCalibration.quality_gain_scale`	0.60–1.60	0.60–1.60	calibrabile (diretto)	`calibrate_twin.py`
`TwinCalibration.energy_scale`	0.60–1.80	0.60–1.80	calibrabile (diretto)	`calibrate_twin.py`
`TwinCalibration.penalty_scale`	0.50–2.50	0.50–2.50	calibrabile (diretto)	`calibrate_twin.py`

3.3 Nota di lettura

Questo capitolo e la baseline ufficiale per calibrazione e audit. Il dettaglio matematico completo resta nei capitoli successivi (Parametrizzazione, Formulazione, Twin, BO).

Capitolo 4

Architettura Software

4.1 Layer applicativi

Layer	File principali	Responsabilita
Knowledge	`core/genetics.py`, `core/literature.py`, `configs/*.json`	Profili genetici, priors cultivar, registry fonti, override bounds/coefficienti.
Model	`core/model.py`	Dataclass dominio, builder profilo strategico, controller adattivo, digital twin, serializzazione profilo/outcome.
Optimization	`optimizer_literature_best.py`	Constrained BO multi-GP con restart robusti e stima incertezza ensemble.
Realtime	`controller_literature_realtime.py`, `core/realtime_io.py`, `core/mpc_supervisor.py`	Ingestione payload multi-source, quality gate, fallback, emissione setpoint e MPC opzionale.
Observability	`core/storage.py`, `core/alerting.py`, `dashboard_timeseries.py`	Persistenza SQLite, alerting rolling-window, dashboard HTML.
Calibration	`calibrate_twin.py`	Fit parametri del twin su cicli osservati (train/val, bootstrap, governance).
Planning	`core/production_planner.py`, `scripts/plan_clone_logistics.py`	Pianificazione logistica multi-ciclo talee con vincoli hard.
Validation	`tests/test_*.py`, `scripts/run_tests.ps1`	Verifica funzionale unit/e2e, sicurezza realtime, vincoli regular, planner.

4.2 Data flow end-to-end

1. caricamento profilo genetico/cultivar e bounds;
2. calibrazione twin opzionale (`calibrate_twin.py`);
3. ottimizzazione profilo con BO constrained;
4. persistenza run optimizer su `optimizer_runs`;
5. ingestione realtime payload sensore;
6. quality gate sensori e gestione fault;
7. correzione euristica setpoint baseline;
8. raffinamento MPC opzionale;
9. emissione `control_tick` su stdout/JSONL/sink attuatore;

10. logging DB e dashboard.

4.3 Mappa zone capannone vs dominio software

Zona business plan	Copertura nel runtime	Note
Zona talee (DWC)	`stage = propagation`	Coerente: propagazione in ambiente separato e conservativo.
Zona vegetativa (NFT)	`stage = vegetative`	Coerente: setpoint vegetativi e gestione nutrizione/traspirazione.
Zona fioritura (NFT/contesto serra)	`stage = flower_early/flower_late`	Coerente sul dominio colturale; il dettaglio serra naturale non e simulato esplicitamente.
Laboratorio + dry room + magazzino	Non modellati in logica di controllo	Trattati come perimetro operativo extra-software.

Capitolo 5

Parametrizzazione Fenologica e Spazio Decisionale (Dettaglio Operativo)

5.1 Fasi fenologiche

Ordine runtime:

`propagation` → `vegetative` → `transition` → `flower_early` → `flower_late`

5.2 Vincoli varietali e produttivi runtime

Il profilo di default e `regular_photoperiodic` con famiglia default `indica_dominant`.
Il `StrategyBuilder` impone durate hard per famiglia.

Famiglia	Prop.	Veg.	Trans.	Flower early	Flower late	Densita fioritura
`indica_dominant`	12	24	7	32	30	4.0 pl/m2
`sativa_dominant`	12	27	7	37	37	2.0 pl/m2
`hybrid`	12	26	7	34	33	3.0 pl/m2

Nel metadata profilo sono registrati anche:

- `business_plan_cuttings_per_plant = 5.0`;
- `business_plan_no_dedicated_mother_plants = True`;
- finestra `seed_to_cutting` hard-coded: 27.0–35.0 giorni.

5.3 Vincoli hard pre-ciclo (seme -> talea)

I parametri pre-ciclo sono trattati come **costanti hard-coded** e non vengono ottimizzati dalla BO. Essi sono usati per governance del profilo e tracciabilita metodologica.

Parametro	Valore hard	Implementazione	Impatto runtime
`SEED_GERMINATION_DAYS_RANGE`	10–14 gg	`core/model.py`	Metadata profilo `business_plan_seed_germination_days_`
`SEEDLING_TO_CUTTING_DAYS_RANGE`	17–22 gg	`core/model.py`	Composizione finestra `seed->talea`.

`business_plan_seed_to_cutting`	27–35	gg	days	`core/model.py` Tracciabilita tecnica del lead time pre-ciclo.

Nota di perimetro: il ciclo simulato dal twin e dal controller realtime parte dalla talea distaccata (`cutting_cycle_start = detached_cutting`); la finestra `seed->talea` e quindi un vincolo hard pre-ciclo, non un intervallo ottimizzabile nel loop giornaliero di controllo.

5.4 Spazio parametrico ottimizzato (literature baseline)

Bounds del costruttore `CannabisYieldLiteratureBuilder`:

Parametro	Min	Max	Unita	Fonte primaria runtime
`veg_ppfd`	360	850	umol m^{-2} s^{-1}	`rodriguez_morrison_2021_light`
`flower_ppfd`	900	1800	umol m^{-2} s^{-1}	`rodriguez_morrison_2021_light`
`flower_photoperiod_h`	12.0	14.0	h	`ahrens_2023_is_twelve_hours`, `ahrens_2024_13h_photoperiod`
`co2_flower_ppm`	450	1200	ppm	letteratura fisiologica CO2
`air_temp_day_c`	23.0	29.0	degC	letteratura CEA cannabis indoor
`rh_day_pct`	50	70	%	modello VPD e rischio patogeni
`ec_veg`	1.4	2.4	mS/cm	`saloner_2020_n`
`ec_flower`	1.8	3.0	mS/cm	`saloner_2019_k`, `powell_2026_mass_balance`
`ph_target`	5.7	6.2	pH	`powell_2026_mass_balance`
`n_mg_l`	140	240	mg/L	`saloner_2020_n`
`p_mg_l`	15	60	mg/L	registry nutrizione runtime
`k_mg_l`	60	240	mg/L	`saloner_2019_k`
`blue_flower_frac`	0.10	0.28	frazione	`holweg_2024_red_white_spectra`
`far_red_flower_frac`	0.02	0.14	frazione	`holweg_2024_red_white_spectra`
`uvb_late_frac`	0.00	0.10	frazione	`llewellyn_2023_uv_minimal_effect`

Nota: le durate fenologiche non sono piu variabili decisionali della BO; il runtime usa solo durate hard family-specific.

5.5 Valori sperimentali assunti: rimando

La baseline pre-calibrazione e formalizzata nel Capitolo 3. In questo capitolo si riportano solo gli elementi necessari al dominio decisionale dell'optimizer e del controller.

5.6 Override genetici/famiglia/cultivar

Precedenza effettiva dei bounds:

`literature_bounds` → `genetic_bound_overrides` → `family_bound_overrides` → `cultivar_bound_overri`

I coefficienti modello sono composti in modo moltiplicativo:

$$c_k = c_k^{base} \cdot c_k^{family} \cdot c_k^{cultivar}$$

con clamp numerici applicati nel twin (`yield_potential_scale`, `quality_potential_scale`, `disease_pressure_scale`, `hlvd_pressure_scale`, `nutrient_window_scale`).

5.7 Evidenze bibliografiche in registry

source_id	Anno	Compat. regular	Citazione sintetica
knight_2010_scrog_hydroponics	2010	no	Knight et al., Forensic Sci Int.
saloner_2019_k	2019	yes	Saloner et al., Front Plant Sci.
saloner_2020_n	2020	yes	Saloner et al., Front Plant Sci.
rodriguez_morrison_2021_light	2021	yes	Rodriguez-Morrison et al., Front Plant Sci.
dowling_2022_photoperiod_meta	2022	yes	Dowling et al., Front Plant Sci.
moher_2021_photoperiod_response_in_vitro	2021	yes	Moher et al., HortScience.
ahrens_2023_is_twelve_hours	2023	yes	Ahrens et al., Plants.
ahrens_2024_13h_photoperiod	2024	yes	Ahrens et al., Plants.
llewellyn_2023_uv_minimal_effect	2023	yes	Rodriguez-Morrison et al., UV-B.
holweg_2024_red_white_spectra	2024	yes	Holweg et al., Front Plant Sci.
powell_2026_mass_balance	2026	yes	Powell et al., Front Plant Sci.
pitman_2020_pythium_report	2020	yes	Pitman et al., Plant Disease.
punja_2025_hlvd_management	2025	yes	Punja, Plants.
ioannidis_2022_ssr_micropropagation	2022	yes	Ioannidis et al., Plants.

Capitolo 6

Formulazione Tecnico-Matematica del Modello

6.1 Variabili derivate

VPD

$$\mathrm{VPD} = e_s(T_{air}) - e_a(T_{air}, RH) \tag{6.1}$$

$$e_s = 0.6108 \exp\left(\frac{17.27T}{T + 237.3}\right), \quad e_a = e_s \frac{RH}{100} \tag{6.2}$$

DLI

$$\mathrm{DLI} = 0.0036 \cdot \mathrm{PPFD} \cdot \mathrm{photoperiod_h} \tag{6.3}$$

6.2 Dinamica giornaliera

Il twin usa aggiornamenti laggati con rumore gaussiano:

$$x_t = \mathrm{clip}(x_{t-1} + \alpha(x^\star - x_{t-1}) + \epsilon_t) \tag{6.4}$$

dove $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$, con α e σ specifici per variabile (`t_air_c`, `rh_pct`, `co2_ppm`, `ppfd`, `t_solution_c`, `ec_ms_cm`, `ph`, `do_mg_l`).

6.3 Funzioni di risposta

$$f_G(x) = \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right), \qquad f_L(x) = \frac{1}{1 + \exp\left(-\frac{x-c}{s}\right)} \tag{6.5}$$

Le funzioni gaussiane/logistiche sono usate per temperatura, VPD, nutrizione, root-zone, CO2 e spettro.

6.4 Pesi fenologici e gain

Fase	Peso resa	Peso qualita	Uso nel modello
`propagation`	0.06	0.00	rooting e stabilizzazione iniziale
`vegetative`	0.32	0.06	espansione canopy
`transition`	0.78	0.20	transizione fisiologica

`flower_early`	1.28	0.75	bulk resa fioritura
`flower_late`	1.06	1.15	maturazione qualita

$$\Delta Y_t = \mathrm{DLI}_t \cdot 0.52 \cdot w_Y(stage) \cdot canopy \cdot \Phi_{growth} \cdot health \tag{6.6}$$

$$\Delta Q_t = w_Q(stage) \cdot \Phi_{spectrum,stress,root,health} \tag{6.7}$$

con scaling da coefficienti genetici/famiglia/cultivar e da `TwinCalibration`.

6.5 Penalty e rischio fitosanitario

Incremento `disease_pressure`:

$$\Delta D_t = 0.020 \max(RH-72,0)+0.090 \max(0.82-VPD,0)+0.070 \max(T_{sol}-22,0)+0.095 \max(7-DO,0) \tag{6.8}$$

Incremento `hlvd_pressure`:

$$\Delta H_t = (0.0018(1-s) + 0.0008\Delta D_t) \cdot scale_{hlvd} \tag{6.9}$$

dove s e `sanitation_level`.
La penalty aggrega eccessi/deficit su VPD, pH, EC, DO e RH in fioritura.

Capitolo 7

Controller Adattivo Closed Loop

7.1 Target VPD fase-specifici

Fase	Target VPD (kPa)	Effetto operativo
`propagation`	0.40–0.80	minimizzazione stress idrico iniziale
`vegetative`	0.80–1.20	sostegno crescita vegetativa
`transition`	1.00–1.35	passaggio fenologico controllato
`flower_early`	1.10–1.45	robustezza set infiorescenze
`flower_late`	1.00–1.35	compromesso resa/qualita/rischio

7.2 Regole di controllo implementate

Azioni principali in `AdaptiveController.adjust()`:

1. `raise_vpd` / `lower_vpd`
2. `dli_recovery`
3. `raise_ph` / `lower_ph`
4. `raise_ec` / `lower_ec`
5. `uptake_balance_high_transpiration` / `uptake_balance_low_transpiration`
6. `cool_root_zone`, `raise_do`
7. `quality_spectral_shift` o `yield_spectral_shift` in `flower_late`

7.3 Interazione con quality-gate e fallback

Pipeline operativa per ciascun tick:

1. parsing payload (`sensor_payload_to_state`);
2. valutazione qualita sensore (`evaluate_sensor_quality`);
3. fallback safe su payload invalidi, stream error/idle, hard fault sensore;
4. enforce limiti hard/rate (`enforce_setpoint_limits`);
5. MPC opzionale (`-use-mpc-supervisor`);
6. emissione `control_tick`.

Capitolo 8

Digital Twin: Dettagli Implementativi

8.1 Ciclo di simulazione

`CEADigitalTwin.simulate_cycle()`:

1. inizializza stato da `propagation`;
2. determina fase attiva per giorno;
3. applica controllo adattivo;
4. avanza stato con `_advance_one_day`;
5. accumula yield/quality/energy/penalty;
6. applica correttivi finali malattia/HLVd + `TwinCalibration` post-ciclo.

8.2 Robustezza numerica e guardrail

- clipping sistematico su variabili fisiche;
- seed espliciti per riproducibilita;
- clamp parametri calibrazione (`clamp_twin_calibration`);
- normalizzazione spettro (`_normalize_spectrum`).

8.3 Parametri di calibrazione twin

Parametro	Min	Max	Ruolo
`yield_gain_scale`	0.60	1.60	scaling gain giornaliero resa
`quality_gain_scale`	0.60	1.60	scaling gain giornaliero qualita
`transpiration_scale`	0.60	1.60	scaling proxy traspirazione
`disease_inc_scale`	0.50	2.00	scala crescita pressione patogena
`hlvd_inc_scale`	0.50	2.00	scala crescita pressione HLVd
`penalty_scale`	0.50	2.50	scala penalty di processo
`energy_scale`	0.60	1.80	scala consumo energetico
`yield_post_scale`	0.70	1.30	scala resa post-ciclo
`quality_post_scale`	0.70	1.30	scala qualita post-ciclo
`quality_post_offset`	-20.0	20.0	offset qualita post-ciclo

8.4 Output strutturato

Output nativo:

- `CycleOutcome` (KPI aggregati + log giornalieri);
- `outcome_to_dict()` con `adjustment_summary` e `last_days`;
- `clone_cycle_derived_metrics()` con KPI annualizzati.

Capitolo 9

Constrained Bayesian Optimization

9.1 Obiettivi e vincoli

Per `max_yield` l'obiettivo primario e max `dry_yield_g_m2`.
Per `max_quality` l'obiettivo primario e max `quality_index` con floor resa `quality_y_min`.
Le varianti `*_energy` mantengono l'obiettivo base e aggiungono vincolo hard energia. In tutte le modalita e attivo il target annuo farm-level `yield_target_annual_kg`. L'ottimizzazione richiede una resa areale annua minima `yield_target_kg_m2_year` per rendere il target raggiungibile con l'area attiva disponibile.

9.2 Violation function

`_literature_violation()` penalizza:

- fioritura sottodimensionata vs fase vegetativa;
- fotoperiodo fuori finestra 12–14h;
- eccessi di P/EC;
- criticita processo/sanitarie via `penalty`, `disease_pressure`, `hlvd_pressure`.

9.3 Modello probabilistico multi-GP

Modelli Gaussian Process allenati:

- `gp_obj`
- `gp_yield`
- `gp_violation`
- `gp_penalty`
- `gp_disease`
- `gp_hlvd`
- `gp_energy`

Kernel comune:

$$K = \texttt{ConstantKernel} \times \texttt{Matern}(\nu = 2.5) + \texttt{WhiteKernel}$$

9.4 Acquisition e fattibilita

Acquisition:

$$A(x) = EI(x) \cdot P_{feasible}(x)$$

dove:

$$P_{feasible}(x) = P(v \leq 0.50) \cdot P(pen \leq 15) \cdot P(dis \leq 3) \cdot P(hlvd \leq 0.08) \cdot P(energy \leq cap) \cdot P(yield \geq floor)$$

I fattori energia/floor resa sono attivati in base alla modalita.

9.5 Strategia restart e robustezza

- `max_yield`: restart multipli + robust score Monte Carlo multi-seed.
- `max_quality`: restart multipli fino a candidato fattibile.
- incertezza finale: ensemble run con statistiche `p05/p50/p95`, `std`, `feasible_rate`.

Capitolo 10

Contratto Dati Realtime e Integrazione I/O

10.1 Campi obbligatori e opzionali payload sensore

Campo	Tipo	Stato
`t_air_c`	float	obbligatorio
`rh_pct`	float	obbligatorio
`co2_ppm`	float	obbligatorio
`ppfd`	float	obbligatorio
`t_solution_c`	float	obbligatorio
`do_mg_l`	float	obbligatorio
`ec_ms_cm`	float	obbligatorio
`ph`	float	obbligatorio
`timestamp`	ISO string	opzionale
`transpiration_l_m2_day`	float	opzionale
`dli_prev`	float	opzionale
`vpd_kpa`	float	opzionale
`disease_pressure`	float	opzionale
`hlvd_pressure`	float	opzionale
`power_total_kw`	float	opzionale (telemetria energetica)
`power_led_kw`	float	opzionale (telemetria energetica)
`power_hvac_kw`	float	opzionale (telemetria energetica)
`power_pumps_kw`	float	opzionale (telemetria energetica)

10.2 Sorgenti e sink supportati

Sorgenti:

- `jsonl_file`
- `stdin_json`
- `http_poll`
- `serial_json`
- `mock_stream`

Sink:

- stdout JSON;
- append su JSONL;

- HTTP POST attuatore;
- seriale attuatore.

10.3 Schema output `control_tick`

Campo	Descrizione
`event`	Evento strutturato (`control_tick`).
`ts`	Timestamp ISO.
`mode`, `cycle_day`, `stage`	Contesto runtime.
`actions`	Lista azioni controller/fallback/MPC.
`sensor`	Stato sensore validato.
`baseline_setpoint`	Setpoint nominale stage.
`heuristic_setpoint`	Output controller euristico pre-limiti/MPC.
`recommended_setpoint`	Setpoint finale emesso.
`sensor_quality`	Score + warning + hard fault.
`energy_observability_kw`	Telemetria potenza opzionale estratta da `power_*` del payload.
`mpc`	Diagnostica MPC (se attivo).
`fault`	Oggetto fault in fallback ticks.
`governance`	`run_id`, `profile_signature`.

Capitolo 11

Runbook Operativo

11.1 Setup ambiente

```
cd regular_clone
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install -r
   requirements_realtime_optional.txt
```

11.2 Calibrazione twin

```
.\.venv\Scripts\python.exe .\calibrate_twin.py ^
  --dataset-json .\calibration_dataset_template.json ^
  --out-json .\runtime\profiles\twin_calibration.json ^
  --store-db .\runtime\db\cea_timeseries.db
```

Il template e self-contained (profili inline) e validato con controlli hard su coerenza famiglia/densita/sottosistema idroponico.

11.3 Ottimizzazione offline

```
.\.venv\Scripts\python.exe .\optimizer_literature_best.py ^
  --mode both_energy --quality-floor 62 --energy-cap-kwh-m2 700 --json-
     only
```

11.4 Realtime

Con auto-generazione profilo:

```
.\.venv\Scripts\python.exe .\controller_literature_realtime.py ^
  --mode max_yield --quality-floor 62 --source serial_json --serial-port
      COM4 --serial-baud 115200 ^
  --watchdog-timeout-s 5 --yield-target-annual-kg 80 --farm-active-area-
     m2 120 ^
  --store-db .\runtime\db\cea_timeseries.db
```

Con vincolo energia nativo in realtime:

```
.\.venv\Scripts\python.exe .\controller_literature_realtime.py ^
  --mode max_yield_energy --quality-floor 62 --energy-cap-kwh-m2 640 ^
  --yield-target-annual-kg 80 --farm-active-area-m2 120 ^
  --source serial_json --serial-port COM4 --serial-baud 115200 ^
  --start-date 2026-04-01
```

Con profilo gia esportato + MPC:

```
.\.venv\Scripts\python.exe .\controller_literature_realtime.py ^
  --mode max_quality --profile-json .\runtime\profiles\
     literature_best_profile_runtime.json ^
  --source mock_stream --use-mpc-supervisor --mpc-horizon 6 --mpc-
     candidates 96
```

11.5 Analisi economica pre-go-live

```
.\.venv\Scripts\python.exe .\scripts\run_economic_analysis.py ^
  --target-annual-kg 80 --price-eur-g 4.0 ^
  --mix-indica 0.5 --mix-sativa 0.5 ^
  --yield-source optimizer --optimizer-mode max_yield_energy ^
  --quality-floor 62 ^
  --infrastructure-profile startup_low_capex ^
  --energy-architecture grid_only_retrofit ^
  --monitoring-tier core_efficiency_extended ^
  --energy-cap-kwh-m2-cycle 700 ^
  --economic-yield-basis-policy target ^
  --out-json .\runtime\logs\economic_analysis_project.json
```

Output principale: KPI annui (ricavi, EBITDA, ROI, payback, break-even) con base economica esplicita. Campi chiave: `target_annual_kg`, `plan_projected_annual_yield_kg`, `economic_yield_basis_pol` `economic_yield_basis_kg`, `target_vs_planned_kg`, `selected_configuration`, `candidate_rankings`, `constraints_satisfied`.

11.6 Baseline startup low-cost ufficiale

Per lo stato attuale pre-calibrazione la baseline economico-tecnica ufficiale e: `startup_low_capex + grid_only_retrofit + core_efficiency_extended`.
Il selettore economico valuta il catalogo completo e seleziona la prima configurazione con `constraints_satisfied.all = true` e `capex_total` minimo.

Elemento	Politica baseline
Energia	rete elettrica + predisposizione retrofit (PV/batteria in fase 2)
Monitoraggio	8 sensori core obbligatori + telemetria energetica opzionale `power_*`
Controllo resa	target annuo in uguaglianza (`yield_target_annual_kg = 80` default)
Controllo energia	cap hard su modalita `*_energy`
Controllo qualita	`quality_floor = 62` su modalita resa
Vincoli fenologici	invariati (durate/densita hard indica/sativa, DWC/NFT)

11.7 Artefatti runtime

Artefatto	Path default
DB SQLite	`runtime/db/cea_timeseries.db`
Output JSONL controllo	`runtime/jsonl/control_output_literature_realtime.jsonl`
Profili runtime	`runtime/profiles/`
Dashboard HTML	`runtime/logs/dashboard_timeseries.html`
Analisi economica JSON	`runtime/logs/economic_analysis_project.json`

Capitolo 12

Validazione e Test

12.1 Strategia test

La validazione copre:

- unit test su modelli, bounds, qualita sensori, alerting;
- test e2e optimizer e calibrazione;
- test safety realtime (fallback + limiti hard/rate);
- test planner logistico multi-ciclo.

12.2 Suite test corrente

Comando ufficiale:

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -p "test_*.py"
    -v
```

File test	Copertura
`test_e2e_optimizer.py`	Run modes optimizer, metriche derivate clone cycle, varianti energy, guardrail `quality_floor`.
`test_e2e_calibration.py`	E2E CLI calibrazione twin su dataset sintetico.
`test_realtime_safety.py`	Fallback setpoint, emissione fallback tick, limiti hard/rate, parsing telemetria `power_*`.
`test_realtime_stream_events.py`	Eventi stream `idle/error/valid`.
`test_sensor_quality.py`	Range hard, jump hard/warning, scoring.
`test_alerting.py`	Trigger/cooldown alert rolling-window.
`test_model_coefficients.py`	Composizione coefficienti base/famiglia/cultivar.
`test_literature_bounds_and_sources.py`	Integrita bounds e registry fonti.
`test_reference_audit_summary.py`	Sintesi compatibilita referenze per profilo.
`test_realtime_mode_support.py`	Coerenza mapping modalita energy in realtime + validazione parsing cap energia e helper runtime.
`test_regular_profile_constraints.py`	Vincoli durate/densita hard regular per famiglia e finestra hard `germinazione/seed->talea`.
`test_production_planner.py`	Vincoli planner 80 kg/anno, check saturazione e match esatto del target annuo da schedule settimanale.

`test_economic_hybrid_model.py`	Verifica modulo economico locale (catalogo configurazioni, selettore min-CAPEX e backward compatibility legacy).

12.3 Criteri minimi di accettazione

1. test suite passata al 100%;
2. nessun hard fault sensore non gestito;
3. fallback e alert persistiti in DB quando `-store-db` e attivo;
4. coerenza KPI optimizer/realtime con vincoli modalita.

Capitolo 13

Sicurezza Operativa

13.1 Guardrail minimi

Guardrail	Implementazione
Fallback recipe conservativo	`safe_fallback_setpoint()` e `emit_safe_fallback_tick()`.
Hard limits setpoint	`enforce_setpoint_limits()` con bounds fisici per clima/nutrizione/spettro.
Rate limits per tick	Clamp variazione per `ppfd`, `t_air_c`, `rh_pct`, `co2_ppm`, `t_solution_c`, `do_mg_l`, `ec_ms_cm`, `ph`.
Quality gate sensori	`evaluate_sensor_quality()` con hard faults e warnings.
Watchdog stream	Trigger fallback su timeout (`-watchdog-timeout-s`).

13.2 Fault handling e policy fallback

Eventi fault principali:

- `safe_fallback_watchdog_timeout`
- `safe_fallback_source_error`
- `safe_fallback_sensor_payload_error`
- `safe_fallback_sensor_quality_fault`

Ogni fallback genera un `control_tick` strutturato con campo `fault`.

13.3 Alerting rolling-window

Parametro	Default	Campo CLI	Significato
Finestra tick	60	`-alert-window-ticks`	ampiezza finestra mobile
Soglia fallback rate	0.15	`-alert-fallback-rate`	`trigger-threshold fallback_rate_high`
Soglia timeout count	3	`-alert-timeout-count`	`trigger-threshold timeout_burst`
Soglia payload errors	3	`-alert-payload-error`	`trigger-count-threshold payload_error_burst`
Soglia quality faults	3	`-alert-quality-fault`	`trigger-count-threshold sensor_quality_fault_burst`

Cooldown	30	`-alert-cooldown-minutes`	Soppressione alert

Capitolo 14

Traceability e Governance

14.1 Matrice codice -> sezioni

Dominio	File fonte di verita	Sezioni documento
Architettura complessiva	`README.md`, entrypoint root, `core/*.py`	Cap. 2–3
Fenologia e bounds	`core/model.py`, `optimizer_literature_best.py`, `configs/genetic_profiles.json`	Cap. 4–5
Controller realtime	`controller_literature_realtime.py`, `core/realtime_io.py`	Cap. 6, 9, 12
MPC supervisor	`core/mpc_supervisor.py`	Cap. 6, 12
BO vincolata	`optimizer_literature_best.py`	Cap. 8
Modalita realtime energy-constrained	`controller_literature_realtime.py`, `core/realtime_io.py`, `core/mpc_supervisor.py`	Cap. 2, 6, 10
Calibrazione twin	`calibrate_twin.py`	Cap. 7, 10
Storage e audit	`core/storage.py`, `core/governance.py`	Cap. 10, 13
Planner logistico	`core/production_planner.py`	Cap. 10, 14
Blocco economico	`economics/cea_economic_analysis.py`	Cap. 2, 11, 15
Validazione	`tests/test_*.py`, `scripts/run_tests.ps1`	Cap. 11

14.2 Run identity e firme

La governance runtime include:

- `run_id` via `new_run_id(prefix)`;
- `git_commit` via `git_commit_short()`;
- firma payload via `dict_signature()` (profili/run/runtime);
- firma file via `file_signature()` usata nel flusso di calibrazione dataset.

14.3 Persistenza auditabile

Schema SQLite inizializzato da `init_storage()`:

- `control_ticks`

- `optimizer_runs`
- `twin_calibrations`
- `alert_events`

Ogni tabella mantiene `payload_json` completo per audit retrospettivo.

Capitolo 15

Allineamento Metodologico al Business Plan

15.1 Trasposizione metodologia produttiva

Mappa sintetica business plan → runtime:

Elemento business plan	Stato runtime	Evidenza
Ciclo da talea	implementato	stage `propagation` + metadata `cuttings_source`.
DWC in radicazione	implementato	`propagation_system = DWC`.
NFT in vegetativa/fioritura con ricircolo	implementato	`production_system = NFT_recirculating`.
Gestione nutrizione (EC/pH/NPK)	implementato	setpoint stage + regole controller + penalty model.
Zonizzazione capannone (talee/veg/fioritura)	implementato a livello logico	mapping stage fenologici.
Dry room/laboratorio/magazzino	non modellato nel controllo	perimetro operativo extra-software.

15.2 Scostamenti espliciti BP vs codice

Tema	Business plan	Runtime `regular_clone`
Piante madri dedicate	previste in avvio processo	`business_plan_no_dedicated_mother_plants=T` e source talee da piante in vegetativa.
Densita impianto	riferimenti testuali 15/9 (e passaggi 16 e 6/9) pl/m2	vincoli hard profilo regular: indica 4, sativa 2, hybrid 3 pl/m2 in fioritura.
Durate fasi iniziali	radicazione circa 15 gg + vegetativa circa 25 gg	vincoli hard pre-ciclo: germinazione 10–14 gg, seed->talea 27–35 gg; da talea a raccolta: 12/24/7/32/30 (indica), 12/27/7/37/37 (sativa).

Contesto serra luce naturale	descritto per risparmio energetico	non modellato come forcing meteorologico/stagionale esplicito.
Zonizzazione non coltiva	dry room/laboratorio/magazzino dettagliati	non presenti nel dominio software di controllo/ottimizzazione.

15.3 Gap metodologici dichiarati

GAP-01: il business plan non fornisce setpoint machine-ready completi per ogni fase (T/RH/CO2/EC/pH/DO).
GAP-02: il business plan non formalizza equazioni digital twin/BO/MPC; la formulazione deriva dal codice implementativo.

15.4 Implicazioni ingegneristiche degli scostamenti

- il runtime e progettato per robustezza software e controllo ripetibile, non per replica letterale della BOM impiantistica;
- le assunzioni hard su ciclo/densita influenzano direttamente KPI annualizzati e piano logistica talee;
- la mancata modellazione meteo-serra suggerisce una roadmap di estensione verso forcing esogeno.

Capitolo 16

Limiti Noti e Roadmap

16.1 Limiti attuali

- modello digital twin semplificato (proxy biologici) e non gemello fisico completo;
- MPC candidate-based campionato (non NMPC continuo con solver dedicato);
- assenza modellazione esplicita condizioni meteo/fotoperiodo locale esterno;
- dipendenza dalla qualita dati nel loop realtime e dataset calibrazione.

16.2 Roadmap tecnica suggerita

1. integrazione forcing meteo/fotoperiodo esterno nel twin e nel controller;
2. estensione multi-sito/multi-lotto per calibrazione;
3. stima online di stato/parametri (state estimation);
4. estensione economico-energetica da caso statico a co-ottimizzazione operativa (oggi il modulo economico e locale ma ancora separato dal loop di controllo).

Capitolo 17

Riferimenti Tecnici

17.1 Fonti runtime e interne

1. Team Progetto CEA (2026). *BUISNESS PLAN (1)*. Documento interno metodologico.
2. `regular_clone/configs/literature_sources.json`. Registry fonti con campo `compatibility_regula`
3. `regular_clone/configs/genetic_profiles.json`. Profili genetici e override bounds/coefficienti.
4. `regular_clone/configs/cultivar_catalog.json`. Priors cultivar e riferimenti associati.

17.2 Letteratura agronomica in registry

1. Knight G. et al. (2010), Forensic Sci Int. 202(1-3):36-44.
2. Saloner A. et al. (2019), Front Plant Sci. 10:1369.
3. Saloner A. et al. (2020), Front Plant Sci. 11:572293.
4. Rodriguez-Morrison V. et al. (2021), Front Plant Sci. 12:646020.
5. Rodriguez-Morrison V. et al. (2021), Front Plant Sci. 12:725078.
6. Dowling A.V. et al. (2022), Front Plant Sci. 12:797425.
7. Moher M. et al. (2021), HortScience 56(2):183-189.
8. Ahrens A. et al. (2023), Plants 12(14):2605.
9. Ahrens C.W. et al. (2024), Plants 13(3):433.
10. Holweg M.M.S.F. et al. (2024), Front Plant Sci. 15:1393803.
11. Powell K. et al. (2026), Front Plant Sci. 16:1753553.
12. Pitman T.L. et al. (2021), Plant Disease 105(4):1230.
13. Punja Z.K. (2025), Plants 14(5):830.
14. Ioannidis K. et al. (2022), Plants 11(19):2569.

17.3 Riferimenti metodologici software/matematici

1. Allen R.G. et al. (1998). FAO56 crop evapotranspiration.
2. Jones D.R. et al. (1998). Efficient Global Optimization.
3. Gelbart M.A. et al. (2014). Bayesian optimization with unknown constraints.
4. Rasmussen C.E., Williams C.K.I. (2006). Gaussian Processes for Machine Learning.
5. Mayne D.Q. (2014). Model Predictive Control.

Appendice A

Esempio payload sensore


```
{
  "timestamp": "2026-03-22T10:15:00",
  "t_air_c": 25.8,
  "rh_pct": 61.0,
  "co2_ppm": 780.0,
  "ppfd": 930.0,
  "t_solution_c": 20.1,
  "do_mg_l": 7.5,
  "ec_ms_cm": 2.0,
  "ph": 5.9,
  "transpiration_l_m2_day": 3.2,
  "dli_prev": 38.5
}
```

Appendice B

Esempio output control_tick


```
{
  "event": "control_tick",
  "ts": "2026-03-22T10:15:01",
  "mode": "max_yield",
  "cycle_day": 44,
  "stage": "flower_early",
  "actions": [
    "dli_recovery",
    "uptake_balance_high_transpiration",
    "enforce_limits",
    "mpc_supervisor"
  ],
  "sensor_quality": {
    "score": 94.0,
    "warnings": [],
    "hard_faults": []
  },
  "governance": {
    "run_id": "rt_20260322101501_ab12cd34",
    "profile_signature": "sha256..."
  }
}
```

Appendice C

Requisiti software

- Python 3.11+
- `numpy >= 2.0.0`
- `scipy >= 1.7.0`
- `scikit-learn >= 1.0.0`
- opzionale realtime seriale: `pyserial >= 3.5`

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pip install -r
   requirements_realtime_optional.txt
```